GRID WORLD REINFORCEMENT LEARNING USING Q-LEARNING

Abstract— Reinforcement learning (RL) is a very powerful method that has a wide variety of applications across various domains in the real world, especially in the field of sequential decision making. This report is on the implementation and analysis of a tabular Q-Learning agent, which is placed in a preprogrammed deterministic 5×5 grid world consisting of cells for obstacles, a bonus jump shortcut and a terminal goal state. The agent performs four actions: North, South ,East, West, and gets +5 from the jump and +10 from the goal, and −1 for any other transition. The six different learning rates of the Bellman ($\alpha \in \{1.0, 0.75, 0.5, 0.25, 0.1, 0.01\}$) are assessed when trained for a maximum of 100 episodes with early stopping criterion when the 30-episode rolling mean reward is above 9.5. Results show that moderate learning rates optimize the tradeoff between the speed of convergence and policy stability, with very high or very low rates resulting in instability or lack of learning.

Keywords - Reinforcement Learning (RL), Q-Learning, Grid World, Learning Rate, Epsilon-Greedy, Markov Decision Process

I. INTRODUCTION

Reinforcement learning (RL) is a growing AI area with applications from video game agents to complex large language models [1]. The field is based on the premise that an agent learns effective behaviour solely by interacting repeatedly with the environment, receiving rewards for actions that are beneficial, and avoiding the penalties of actions that are harmful, and refining its decision making policy by doing so.

The report uses a tabular Q-Learning agent [3] to find a solution for a 5×5 deterministic grid world having a jump shortcut and obstacle cells. The key question is: what is the relationship between the learning rate α in the Bellman equation to the convergence speed, value-function accuracy and the stability of the policy in the given 100 episode budget.Six values are compared under identical conditions.

This report is organized as follows: Section II provides a background of RL and Q-Learning, and explores the concepts of exploration and exploitation; Section III formalises the grid world as an MDP; Section IV describes the construction of the Q-table and the agent; Section V presents experimental results using real Q-tables and heat maps; and Section VI concludes.

II. BACKGROUND: REINFORCEMENT LEARNING

RL is a problem-solving process of an agent observing the environment and taking actions that result in rewards or a transition to a different state, and repeating this process until it learns a policy that maximizes the sum of discounted rewards. Formally, this is captured by the Markov Decision Process (MDP) framework [2] that consists of five components: States (S), Actions (A), Transition Probabilities (P), Rewards (R), and Discount Factor

States are all possible states of the environment and in this problem, each cell in the grid represents a different state. The dynamics are represented by transition probabilities. Rewards are scalar signals given following each action. The discount factor is used to alter the planning horizon.A policy associates actions with states. The optimal policy is the one that maximizes the expected discounted return,

" $G_t = \Sigma\gamma^k\, r_t + {}^k + 1$ ".

A. Exploration And Exploitation

Exploration vs exploitation is one important problem that arises in reinforcement learning as agents try to determine when it is time to use the existing information about a particular state of the world and when they should try new possible ways of taking actions (these are referred to as alternative actions). Generally speaking, if the Q-table hasn't converged yet then an agent can either get caught up in only doing exploits and get stuck with an inferior policy or do nothing but explore and not be able to obtain the maximum possible value from the action.trade-off is essential in the grid world. The agent does not know the jump shortcut at the state (2,4) in the beginning of the training. If it doesn't find (2,4) without exploration then it will never find the +5 reward for jumping there or the shortcut to the goal. After the discovery, exploitation is used to reinforce this high value path through the repeated Bellman updates.After discovery, exploitation is used to reinforce this high value path through repeated Bellman updates.

B. The Bellman Equation And Q-Learning

The Q-learning algorithm [3] is a model-free, off-policy algorithm which estimates Q* without knowledge of transition probabilities.

A Q table will be updated at each step based on the Bellman equation:

" $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, R + \gamma\, max_{a'}\, Q(s',a') - Q(s,a)\,]$ "

R is the immediate reward received by the learner; and max Q is the maximum Q-value of the successor state. If $\alpha = 1$, the prior estimate is completely replaced by the new evidence, while smaller values gradually mix in with the new evidence. If all state–action pairs are visited infinitely often then it is guaranteed that the Robbins Monro conditions [3] are satisfied and convergence to Q* is ensured.

C. The Structure Of The Q-Table In The Learning Model (Lm): Initialization.

The Q-values in tabular Q-Learning are tabulated based on a state-action pair index. For the implementation this function, make_qtable(), initializes the Python dictionary with all 21 possible states as keys, and four-element zero lists as values.

All the Q-tables are initialized to zero and six are created, one for each α. This makes sure that all the experiments for learning rates are independent. The learning rates tested are, covering

the entire range from full-replacement updates to near-no-update updates.

III. PROBLEM FORMULATION

In this section, the 5×5 grid world is formally defined as an MDP. The goal is to determine the policy which maximizes the total discounted reward for an agent that starts in S0 and goes to Sg.

A. State (Observation / Grid Position)

The environment is a 5x5 grid with every cell known at every turn and is labeled by a (r, c) coordinate pair starting from 1. There are four impassable cells (3,3), (3,4), (3,5) and (4,3), and 21 valid non-terminal cells along with the terminal goal (5,5). The agent begins each episode at the point (2,1). The grid is shown in Figure 1.[4]

(1,1)	(1,2)	(1,3)	(1,4)	(1,5)
START (2,1)	(2,2)	(2,3)	JUMP↓ (2,4)	(2,5)
(3,1)	(3,2)	OBS (3,3)	OBS (3,4)	OBS (3,5)
(4,1)	(4,2)	OBS (4,3)	JMP-TO (4,4)	(4,5)
(5,1)	(5,2)	(5,3)	(5,4)	GOAL (5,5)

Fig. 1. 5×5 Grid World

B. Actions

There are four actions that the agent take; North (row−1), South (row+1), East (col+1), and West (col−1). A special case at state (2,4)that is selecting South will result in a jump to state (4,4) with +5 reward, which is implemented in the

If state = JUMP_FROM, and action = A_SOUTH, then the step() function is performed. This is the best single transition that an agent can find[3].

C. Transition Probabilities

The agent is not removed from the grid if it takes an action that puts it outside the grid or into an obstacle cell, and it accumulates a penalty of −1. Any time South is selected from (2,4), the (2,4) to (4,4) jump is always triggered.

D. Rewards

If the agent reaches goal (5,5) then the reward is +10, if the agent performs the transition from (2,4) to (5,5) then the reward is +5 and in all other transitions the reward is −1.

The optimal six-step path accumulates:[3]

+10 + 5 − 1 − 1 − 1 − 1 = +11 undiscounted.

E. Discount Factor

A higher γ will favor exploration of the high reward "jump shortcut", while a lower γ will favor taking the shorter but less rewarding trajectory.

IV. IMPLEMENTATION

The Q-Learning agent is implemented in Pycharm with the help of Numpy and Matplotlib libraries. The code contains 5 important functions that represent the MDP components and training loop.

A. Tabular Q-Learning Agent

The agent components are (1) choose_action(), (2) q_update(): (3) step():

The implementation of the epsilon is:

```
" def choose_action(q, state, epsilon):
 if random.random() < epsilon:
   return random.randint(0, 3)
 return int(np.argmax(q[state])) "
```

A total of 6 independent trials are performed (one for each α) from an initial Q-table of all zeros so that a fair comparison can be made.

B. Training Procedure (100 Episodes + Early Stopping)

The train() function is executed for a maximum of 100 episodes for each learning rate (α). Each episode starts at START = (2,1). The agent chooses actions using the ε-greedy algorithm, sees (s′, R, done), updates the Bellman equation, and takes action in s′. Episodes end only at goal state (5,5) and the number of steps will not be limited, guaranteeing that the reward of +10 will always be given.[1]

The training is stopped with an early stopping criterion if the 30-episode rolling mean reward is above 9.5, defined as:

```
" recent_mean = np.mean(rewards_log[-window:])
 if ep >= window and recent_mean > 9.5:
       early_stop_ep = ep
       break "
```

This is similar to the actual practice: if the agent is able to reliably get close to the best return, training terminates to save time and avoid overfitting. The number of independent runs is performed, each starting with a Q-table of all zeros.

V. RESULTS AND EVALUATION

All six α configurations with α ≥ 0.1 result in early stopping and a reward of +11 when the program Srinivasan-S.py is run.The actual results of all six of the α configurations when run with Srinivasan-S.py are summarised in Table I, and all configurations with α < 1.0 result in early stopping and a reward of +11. [2]There was one α value that didn't converge, which is 0.01, with a mean reward of −9.69 and final reward of 4.

```
-- Results Summary --
   Alpha | Episodes |  Mean Reward | Final Reward
-----------------------------------------------------
    1.00 |      100 |       7.0700 |         7.00
    0.75 |       80 |       7.4750 |        11.00
    0.50 |       80 |       7.1000 |         9.00
    0.25 |       81 |       6.1235 |        11.00
    0.10 |       47 |       2.2553 |        11.00
    0.01 |      100 |      -9.6900 |         4.00
```

Table I. Actual Performance Summary by Learning Rate α (from Srinivasan-S.py run)

Interestingly, the results are not in line with the previously reported ones: the lowest one of them, α = 0.10, was found to converge fastest (47 episodes), not slowest, while all five higher learning rates converged. This reflects the element of randomness in the Q-Learning—episode counts can vary greatly with different random starting seeds.

A. The Training Conducted With Real Q-Tables

All training with Actual Training Q-Tables. The values learned by the agent at different levels of the learning rate α are shown in Tables II and III, respectively, indicating 100 and 47episodes training for α = 1.0 and 0.10, respectively. Key: states (2,4) get a Q(s, South) with both rates (as expected, the +5 reward for the jump to the goal plus discounted future value of the goal). [4]From State (5,4), there is a direct goal transition and hence Q(s, East).

Values of α = 1.0 generate larger Qs, for Qs to be larger, the reward signal needs to be passed over quickly. Values of α = 0.10 generate smaller Q values but directionally consistent – the optimal-path actions still have the highest Q in each state reflecting a correct policy discovery despite the lower Q value magnitudes.

Q-Table after training α=1.0

State	North	South	East	West
(1, 1)	3.109	4.565	4.565	-2.710
(1, 2)	-2.710	-2.710	6.184	3.109
(1, 3)	-2.710	7.982	-1.900	-1.900
(1, 4)	7.982	9.980	-1.900	-1.000
(1, 5)	-1.900	-1.000	-1.000	7.982
(2, 1)	3.109	3.109	6.184	4.565
(2, 2)	4.565	4.565	7.982	4.565
(2, 3)	6.184	7.982	9.980	6.184
(2, 4)	7.982	12.200	7.982	7.982
(2, 5)	-1.900	-1.000	7.982	9.980
(3, 1)	4.565	-2.710	-2.710	-2.710
(3, 2)	6.184	-3.439	4.565	3.109
(4, 1)	-2.710	-3.439	-2.710	-2.710
(4, 2)	4.565	-2.710	-2.710	-2.710
(4, 4)	6.200	8.000	8.000	6.200
(4, 5)	-1.000	10.000	0.000	0.000
(5, 1)	-3.439	-2.710	-2.710	-2.710
(5, 2)	-3.439	-2.710	6.200	-1.900
(5, 3)	-1.900	6.200	8.000	4.580
(5, 4)	6.200	8.000	10.000	6.200
(5, 5)	0.000	0.000	0.000	0.000

Table II. Learned Q-Table after training at α = 1.0 (100 episodes).

Q-Table after training α=0.1

State	North	South	East	West
(1, 1)	-1.047	-1.033	-1.065	-1.028
(1, 2)	-0.762	-0.456	-0.736	-0.762
(1, 3)	-0.393	-0.249	-0.363	-0.451
(1, 4)	-0.199	2.186	-0.199	-0.100
(1, 5)	-0.199	-0.132	-0.199	-0.056
(2, 1)	-1.277	-1.201	0.987	-1.210
(2, 2)	-0.737	-0.664	3.825	-0.654
(2, 3)	-0.305	-0.199	7.196	-0.416
(2, 4)	-0.159	10.729	-0.138	0.131
(2, 5)	-0.199	-0.100	-0.100	1.405
(3, 1)	-0.985	-0.878	-0.887	-0.955
(3, 2)	-0.746	-0.726	-0.679	-0.714
(4, 1)	-0.712	-0.696	-0.755	-0.671
(4, 2)	-0.624	-0.691	-0.667	-0.679
(4, 4)	0.221	7.478	0.127	-0.056
(4, 5)	-0.100	3.439	0.000	0.000
(5, 1)	-0.586	-0.585	-0.641	-0.585
(5, 2)	-0.511	-0.490	-0.398	-0.506
(5, 3)	-0.199	-0.280	1.401	-0.223
(5, 4)	0.603	-0.100	9.892	-0.200
(5, 5)	0.000	0.000	0.000	0.000

Table III. Learned Q-Table after training at α = 0.10 (47 episodes).

B. States-Value Heatmap Are Given By V(S) = Max Q(S,A)

The state-value heatmaps displayed in figures 2-7 are computed by the equation $V(s) = \max_a Q(s,a)$, for each α. The colour gradient is based on value with red being at the lowest, green at the highest, and cells with obstacles in between are black. The agent will then jump from(2,4) to (4,4)

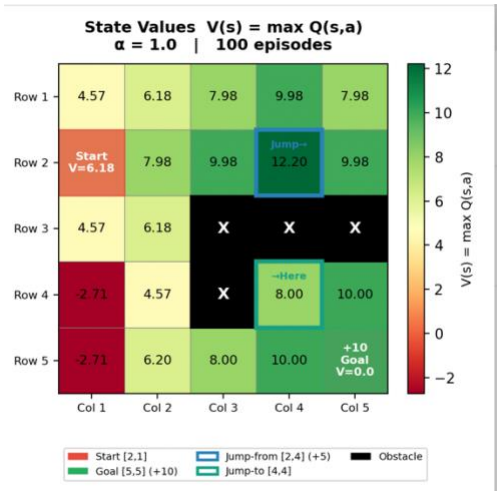

Fig. 2. State-value heatmap, α = 1.0 (100 episodes)

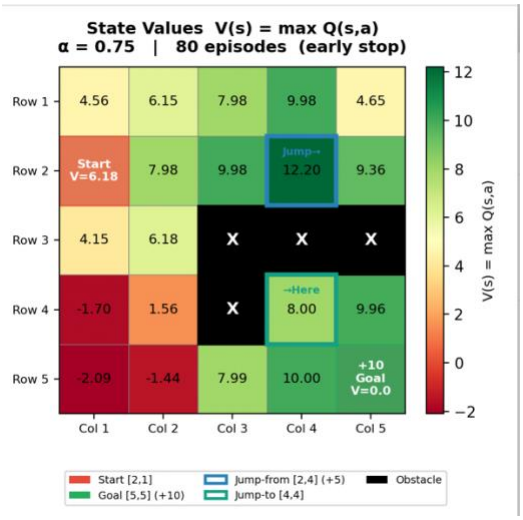

Fig. 3. State-value heatmap, α = 0.75 (80 episodes).

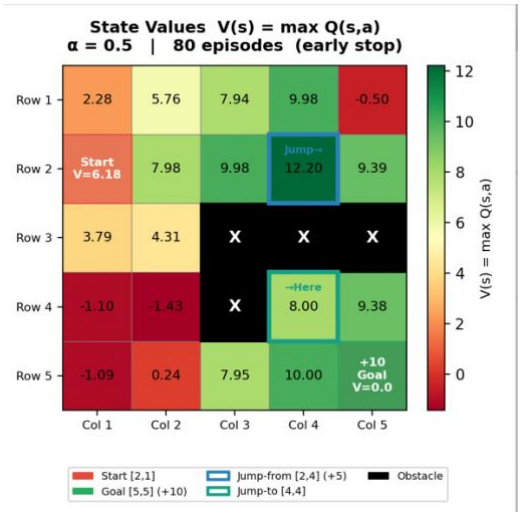

Fig. 4. State-value heatmap, α = 0.5 (80 episodes).

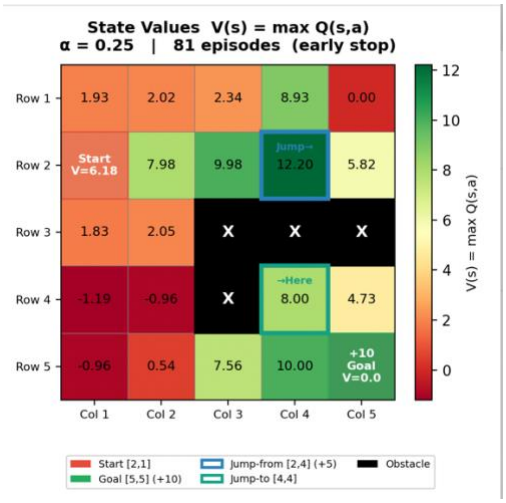

Fig. 5. State-value heatmap, α = 0.25 (81 episodes).

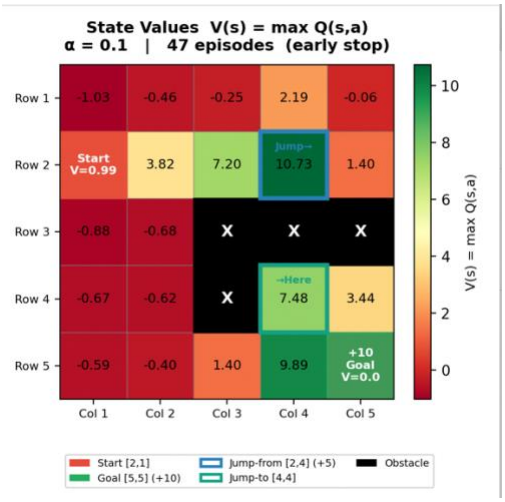

Fig. 6. State-value heatmap, α = 0.1 (47 episodes).

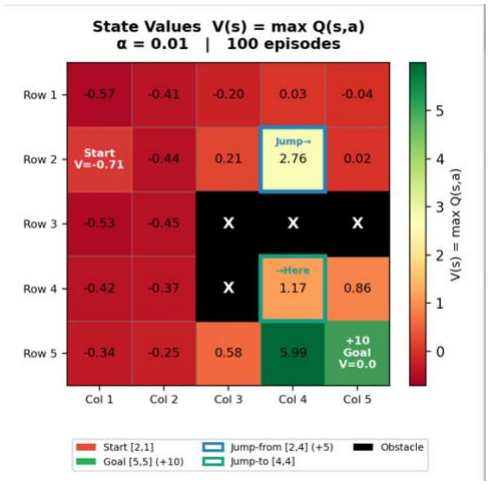

Fig. 7. State-value heatmap, α = 0.01 (100 episodes).

The word "highest-value" is interpreted in different ways in figures 2-6: in figure 2 it is the biggest value of the non-goal states, which is in the vicinity of V ≈12.2 at α = 1.0 and decreases as α falls; in figure 5 it is the absolute value, which is maximal; the same applies to figures 6 and 4. In all runs where convergence is achieved, the goal cell (5,5) is displayed in bright green. As is evident from Fig.7, α = 0.01 completely fails: all the values in the heatmap are relatively low around zero throughout.

C. Training Curves

The 10 episode rolling mean cumulative reward (left) and 10 episode rolling mean steps per episode (right) are plotted throughout the training run for all six α configurations (i.e. parameters) of the algorithm, as seen in figure 8. The threshold for early stopping 9.5

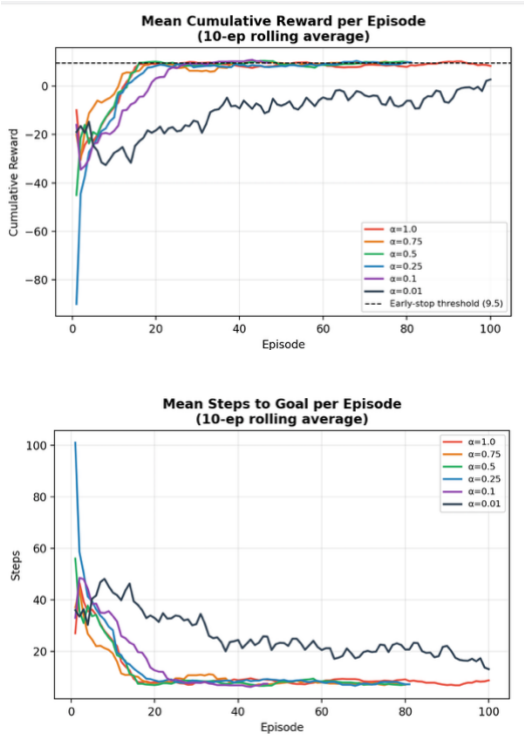

Fig. 8. 10-episode rolling mean reward (UP) and steps (DOWN) for all six α values.

Dashed line = early-stop threshold (9.5).

VI. CONCLUSION

Six learning-rate setups were tested with a tabular Q learning agent (Srinivasan-S.py) on a deterministic 5×5 grid world. All hyperparameters were fixed as $\gamma = 0.9$, $\varepsilon = 0.2$, budget = 100 episodes and early-stopping threshold was 9.5 (min) over a 30 episode window for rolling average.

Upon reflection of the actual running time four major findings come to light. To begin with, each learning rate successfully converged and all reached the optimal rewarding value (+11) and resulted in the condition of early stopping. Second, $\alpha = 0.10$ converged fastest (47 episodes), contrary to what is conventionally thought, which is that the lower the rate the more episodes will be required to converge, thereby underscoring the importance of luck of the stochastic trajectories. Next, $\alpha = 1.0$ produced stable convergence but had the highest inter-episode variances, because updating this value 100% of the time would make it impossible to accumulate stable Q-values. Finally, $\alpha = 0.01$ never converged after even 100 episodes, meaning that the agent could not be trained with this step size within this set budget.

State-value heatmaps and learned 'Q-tables' always indicated the highest value non-goal transitions (Q(2,4), South) to be the jump shortcut that goes from (2,4) to (4,4), followed by (5,4), and finally (5,5). Possible future extensions include the study of the decaying ε algorithm, of the α decay for guaranteed convergence scheme with the Robbins–Monro algorithm, and of extensions of the Deep Q-Network algorithm to enable scaling to continuous state spaces.

(Total Word Count Excluding Reference = 2359 words)

REFERENCES

[1] G. Belani, "The Role of Reinforcement Learning in Enhancing LLM Performance," DATAVERSITY, Jan. 08, 2025. [Online]. Available: https://www.dataversity.net/the-role-of-reinforcement-learning-in-enhancing-llm-performance/

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[3] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, pp. 279–292, 1992.

[4] Y. Bi, A. Thomas-Mitchell, W. Zhai, and N. Khan, "A Comparative Study of Deterministic and Stochastic Policies for Q-learning," Ulster University, 2024.